

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. **Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.**

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

if  $n \leq 0$  then
  | return 0
else
  |  $k = 1$ 
  | for  $i = 1, \dots, n$  do
  |   |  $k = 2 * k$ 
  |   | MYSTERY2( $k$ )
  | return MYSTERY1( $n - 1$ ) +  $k$ 

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  | return 1
else
  | return MYSTERY2( $n/2$ ) + MYSTERY2( $n/2$ ) +  $n^2 + n + 1$ 

```

Soluzione. Analizziamo prima il costo di MYSTERY2. Tale funzione richiama se stessa due volta su input $n/2$ e le altre operazioni nella chiamata hanno costo costante. L'equazione di ricorrenza di MYSTERY2 è quindi:

$$T'(n) = \begin{cases} 1 & n \leq 0 \\ 2T'(n/2) + 1 & n > 0 \end{cases}$$

Tale equazione di ricorrenza può essere risolta con il Master Theorem:

$$\alpha = \log_2 2 = 1 > 0 = \beta \Rightarrow T'(n) = \Theta(n^\alpha) = \Theta(n)$$

La complessità di MYSTERY1 dipende dal ciclo for e dalla chiamata ricorsiva nell'ultima riga. Il ciclo for viene eseguito n volte. In ogni iterazione il valore k , inizialmente 1, viene raddoppiato ed utilizzato come input per una chiamata alla funzione MYSTERY2. Poiché la funzione MYSTERY2 ha un costo lineare il costo complessivo del ciclo for è:

$$\Theta(2) + \Theta(4) + \dots + \Theta(2^n) = \sum_{i=1}^n \Theta(2^i) = \Theta\left(\sum_{i=1}^n 2^i\right) = \Theta\left(\frac{2^{n+1} - 2}{2 - 1}\right) = \Theta(2^{n+1})$$

L'equazione di ricorrenza di MYSTERY1 è quindi:

$$T(n) = \begin{cases} 1 & n \leq 0 \\ T(n-1) + 2^n & n > 0 \end{cases}$$

Tale equazione di ricorrenza può essere risolta col metodo iterativo.

$$\begin{aligned} T(n) &= T(n-1) + 2^n \\ &= T(n-2) + 2^{n-1} + 2^n \\ &= \dots \\ &= T(n-i) + \sum_{k=n-i+1}^n 2^k \end{aligned}$$

La ricorsione termina quando $n-i=0 \Rightarrow i=n$. Sostituendo $i=n$ nell'equazione sopra otteniamo

$$T(n) = T(0) + \sum_{k=1}^n 2^k = 1 + \frac{2^{n+1} - 2}{2 - 1} = \Theta(2^{n+1})$$

2. Considerare un array di interi $A = [x_1, \dots, x_n]$ per cui è noto a priori che l'array contiene unicamente $0 \leq N \leq n$ valori distinti. Ad esempio:

$A = [2, 3, 3, 3, 1, 1, 3, 1]$ contiene $n = 8$ interi ma solo $N = 3$ valori distinti, gli interi 1, 2, 3.

Vogliamo implementare un algoritmo di ordinamento che, dato in input $A[1, \dots, n]$ e N , utilizzi l'informazione relativa al numero di interi distinti nell'array per velocizzare il tempo di calcolo

- a) Scrivere lo pseudocodice di un algoritmo di ordinamento basato sul seguente approccio:
 1. Identificare gli N interi distinti in A e salvarli in un array ausiliario $B[1, \dots, N]$.
 2. Durante la costruzione di B utilizzare una Tabella Hash H per memorizzare le coppie (num, cnt) , dove num è il valore chiave e cnt è il numero di occorrenze di num in A . La Tabella Hash H può essere utilizzata per verificare che in B non ci siano duplicati.
 3. Ordinare l'array B usando l'algoritmo Mergesort.
 4. Per ogni intero nell'array ordinato B , usare il numero di ripetizioni cnt salvato in H per costruire un array A ordinato.
- b) Analizzare la complessità nel caso medio dell'algoritmo proposto in termini della lunghezza dell'array n e del numero di valori distinti N .

Soluzione.

- a) L'algoritmo di ordinamento proposto può essere descritto dal seguente pseudocodice:
- b) Il costo medio delle operazioni SEARCH e INSERT su Hash Table è $O(1)$. Quindi:
 - Step 1 e 2: complessivamente il costo medio è $\Theta(n)$
 - Step 3: il costo ottimo/medio/pessimo di MERGESORT è $\Theta(N \log N)$
 - Step 4: complessivamente i due loop for vengono eseguiti n volte quindi il loro costo ottimo/medio/pessimo è $\Theta(n)$

Mettendo tutto insieme il costo totale nel caso medio è:

$$\Theta(n) + \Theta(n) + \Theta(N \log N) = \Theta(n + N \log N)$$

Algorithm 2: REPSORT(INT $A[1, \dots, n]$, INT N)

```

Let  $B[1, \dots, N]$  be a new array //  $B$  stores the  $N$  distinct integers
 $H = \text{HashTab}()$  //  $H$  stores the number of repetitions of each integer
// Steps 1 and 2: setup  $B$  and  $H$ 
 $k = 1$ 
for  $i = 1, \dots, n$  do
     $\text{cnt} = \text{SEARCH}(H, A[i])$ 
    if  $\text{cnt} \neq \text{NIL}$  then
        |  $\text{INSERT}(H, A[i], \text{cnt} + 1)$  // Increase the count of  $A[i]$ 
    else
        |  $B[k] = A[i]$  // Save  $A[i]$  in  $B$ 
        |  $k = k + 1$ 
        |  $\text{INSERT}(H, A[i], 1)$  // Save  $A[i]$  in  $H$  with starting count 1
// Step 3: sort  $B$ 
MERGESORT( $B, 1, N$ )
// Step 4: construct sorted  $A$ 
 $k = 1$ 
for  $i = 1, \dots, N$  do
     $\text{cnt} = \text{SEARCH}(H, B[i])$ 
    for  $j = 1, \dots, \text{cnt}$  do
        |  $A[k] = B[i]$ 
        |  $k = k + 1$ 

```

3. Dato un array $A[1..n]$ tale che il primo valore è inferiore rispetto all'ultimo, cioè $A[1] < A[n]$, esiste sicuramente un indice $i \in [1, n-1]$ tale che $A[i] < A[i+1]$. Progettare un algoritmo di tipo divide et impera che, dato un array $A[1..n]$ con $A[1] < A[n]$, restituisce un indice i tale che $A[i] < A[i+1]$.

Soluzione.

Si può procedere utilizzando un approccio divide et impera partizionando di volta in volta l'array in due metà.

Si utilizza una funzione ricorsiva che cerca l'indice i considerando un sottoarray che va dall'indice s (start) all'indice e (end). Inizialmente si considera l'intero array, quindi $s = 1$ e $e = n$ e sappiamo che il primo valore è inferiore all'ultimo, ovvero $A[s] < A[e]$. Tale funzione inizialmente verifica se il valore nella posizione intermedia $m = \lfloor (s+e)/2 \rfloor$ soddisfa la condizione $A[m] < A[m+1]$. In caso positivo si restituisce immediatamente m . In caso negativo, si può procedere ricorsivamente alla ricerca in una delle due metà che continua a soddisfare la condizione che il primo valore è inferiore rispetto all'ultimo. Se $A[s] < A[m]$, si procede considerando solo la prima metà che va da s a m visto che in questa metà il primo elemento è inferiore all'ultimo. In caso contrario, avremo $A[m] \leq A[s]$ (negazione di $A[s] < A[m]$) che combinato con l'assunzione $A[s] < A[e]$ implica $A[m] < A[e]$; quindi si può procedere considerando la seconda metà che va da m a e visto che in questa metà il primo elemento è inferiore all'ultimo.

Tale algoritmo viene descritto in pseudocodice in Algorithm 3. Il caso ottimo si verifica quando alla prima chiamata si trova già l'indice da restituire (costo costante $O(1)$). Nel caso pessimo, l'algoritmo effettuerà $\log n$ chiamate ricorsive in quanto ad ogni chiamata dimezza la dimensione della partizione analizzata. Il costo computazionale risulta quindi essere $O(\log n)$.

4. Si consideri il grafo rappresentato dalla seguente matrice di adiacenze:

	A	B	C	D
A	$+\infty$	-10	12	18
B	$+\infty$	$+\infty$	15	9
C	$+\infty$	$+\infty$	$+\infty$	16
D	2	$+\infty$	$+\infty$	$+\infty$

Si esegua manualmente l'algoritmo di Floyd–Warshall descrivendo l'evoluzione della matrice delle distanze durante le iterazioni dell'algoritmo.

Algorithm 3: CERCAINDICE(NUMBER $A[1..n]$) $\rightarrow$ INT

```

return CERCA( $A, 1, n$ )

/* funzione di ricerca dell'indice  $i$  tale che  $A[i] < A[i + 1]$  */
function CERCA(NUMBER  $A[1..n]$ , INT  $s$ , INT  $e$ )  $\rightarrow$  INT
INT  $m \leftarrow \lfloor (s+e)/2 \rfloor$ 
if  $A[m] < A[m + 1]$  then
  | return  $m$ 
else
  | if  $A[s] < A[m]$  then
    | return CERCA( $A, s, m$ )
  | else
    | return CERCA( $A, m, e$ )
  |

```

Soluzione.

La matrice con le distanze viene inizializzata considerando distanza 0 fra un vertice e se stesso e considerando gli archi presenti nel grafo.

	A	B	C	D
A	0	-10	12	18
B	$+\infty$	0	15	9
C	$+\infty$	$+\infty$	0	16
D	2	$+\infty$	$+\infty$	0

La prima iterazione considera la possibilità di usare il vertice A come vertice intermedio.

	A	B	C	D
A	0	-10	12	18
B	$+\infty$	0	15	9
C	$+\infty$	$+\infty$	0	16
D	2	-8	14	0

La seconda iterazione considera anche il vertice B .

	A	B	C	D
A	0	-10	5	-1
B	$+\infty$	0	15	9
C	$+\infty$	$+\infty$	0	16
D	2	-8	7	0

La terza iterazione considera anche il vertice C (si noti che la matrice delle distanze rimane inalterata).

	A	B	C	D
A	0	-10	5	-1
B	$+\infty$	0	15	9
C	$+\infty$	$+\infty$	0	16
D	2	-8	7	0

Ed infine si considera anche il vertice D .

	A	B	C	D
A	0	-10	5	-1
B	11	0	15	9
C	18	8	0	16
D	2	-8	7	0